Archivos .h (proyecto final)

bandit.h

```
#ifndef BANDIT_H
#define BANDIT_H

#include "entity.h"

class Bandit : public Entity
{
public:
    Bandit(Entity* target); // Necesita saber a quién perseguir
    void update(double dt) override;

private:
    Entity* targetPlayer; // Referencia al jugador
    double speed;
};

#endif // BANDIT_H
```

entity.h

```
#ifndef ENTITY_H
#define ENTITY_H

#include <QGraphicsPixmapItem>

class Entity : public QGraphicsPixmapItem
```

```
{
public:
    Entity(QGraphicsItem *parent = nullptr);

    // Métodos para manejar movimiento
    virtual void update(double dt); // Se llamará cada frame

    void setVelocity(double dx, double dy);
    double getVx() const { return vx; }
    double getVy() const { return vy; }

protected:
    double vx; // Velocidad en X
    double vy; // Velocidad en Y
};

#endif // ENTITY_H
```

game.h

```
#ifndef GAME_H
#define GAME_H

#include <QObject>
#include <QGraphicsScene>
#include <QTimer>
#include "levelbase.h"

class Game : public QObject
```

```
{
    Q_OBJECT
public:
    explicit Game(QObject *parent = nullptr);
    QGraphicsScene* getScene() { return scene; }
    void start();

    // Método para cambiar de nivel
    void switchLevel(int levelId);

public slots:
    void gameLoop();

private:
    QGraphicsScene *scene;
    QTimer *timer;
    LevelBase *currentLevel;
};

#endif // GAME_H
```

gamewindow.h

```
#ifndef GAMEWINDOW_H
#define GAMEWINDOW_H

#include <QMainWindow>
#include <QGraphicsView> // IMPORTANTE: Soluciona el error "incomplete type"
```

```
class Game; // Declaración adelantada de la clase Game

class GameWindow : public QMainWindow
{
    Q_OBJECT

public:
    GameWindow(QWidget *parent = nullptr);
    ~GameWindow();

private:
    // Estas son las variables que el compilador te dice que faltan:
    QGraphicsView *view;
    Game *game;
};

#endif // GAMEWINDOW_H
```

lander.h

```
#ifndef LANDER_H
#define LANDER_H

#include "entity.h"
#include <QKeyEvent>

class Lander : public Entity
{
public:
```

```
    Lander();
    void update(double dt) override;

    // Controles
    void keyPressEvent(QKeyEvent *event) override;
    void keyReleaseEvent(QKeyEvent *event) override;

    double getSpeedY() const { return vy; }
    double getFuel() const { return fuel; }

private:
    // Gráficos
    QGraphicsPixmapItem* flame; // El sprite de la llama

    // Estados
    bool rotatingLeft;
    bool rotatingRight;
    bool thrusting;

    // Físicas
    double rotationSpeed;
    double thrustPower;
    double gravity;
    double fuel;

    // NUEVO: Límite de velocidad
    double maxSpeed;
};
```

```
#endif // LANDER_H
```

level1.h

```
#ifndef LEVEL1_H
#define LEVEL1_H

#include "levelbase.h"
#include "playermaraton.h"

class Level1 : public LevelBase
{
public:
    Level1(QGraphicsScene* scene, Game* game); // Constructor actualizado

    void init() override;
    void update(double dt) override;

private:
    PlayerMaraton* player;
    QGraphicsPixmapItem* background;
    QGraphicsPixmapItem* finishFlag; // La meta
};

#endif // LEVEL1_H
```

level2.h

```
#ifndef LEVEL2_H
#define LEVEL2_H

#include "levelbase.h"
#include "playercaravan.h"
#include "bandit.h"

class Level2 : public LevelBase
{
public:
    Level2(QGraphicsScene* scene, Game* game);
    void init() override;
    void update(double dt) override;

private:
    PlayerCaravan* player;
    Bandit* enemy;
    QGraphicsPixmapItem* market; // Meta
    QGraphicsPixmapItem* background;
};

#endif // LEVEL2_H
```

level3.h

```
#ifndef LEVEL3_H
#define LEVEL3_H

#include "levelbase.h"
#include "lander.h"

class Level3 : public LevelBase
{
public:
    Level3(QGraphicsScene* scene, Game* game);
    void init() override;
    void update(double dt) override;

private:
    Lander* player;
    QGraphicsPixmapItem* landingZone; // El objetivo
    QGraphicsPixmapItem* background;

    bool levelFinished;
};

#endif // LEVEL3_H
```

levelbase.h

```
#ifndef LEVELBASE_H
#define LEVELBASE_H

#include <QGraphicsScene>

class Game; // Declaración adelantada

class LevelBase
{
public:
    // Ahora pedimos el puntero a 'Game' en lugar de solo 'Scene'
    LevelBase(QGraphicsScene* scene, Game* game);
    virtual ~LevelBase();

    virtual void init() = 0;
    virtual void update(double dt) = 0;
    virtual void clear();

protected:
    QGraphicsScene* scene;
    Game* game; // Referencia al controlador principal
};

#endif // LEVELBASE_H
```

playercaravan.h

```
#ifndef PLAYERCARAVAN_H
#define PLAYERCARAVAN_H

#include "entity.h"
#include <QKeyEvent>

class PlayerCaravan : public Entity
{
public:
    PlayerCaravan();
    void update(double dt) override;
    void keyPressEvent(QKeyEvent *event) override;
    void keyReleaseEvent(QKeyEvent *event) override;

private:
    double speed;
    bool up, down, left, right;
};

#endif // PLAYERCARAVAN_H
```

playermaraton.h

```
#ifndef PLAYERMARATON_H
#define PLAYERMARATON_H

#include "entity.h"
#include <QKeyEvent>

class PlayerMaraton : public Entity
{
public:
    PlayerMaraton();

    void update(double dt) override;

    void keyPressEvent(QKeyEvent *event) override;
    void keyReleaseEvent(QKeyEvent *event) override;

private:
    // Variables de movimiento
    bool movingLeft;
    bool movingRight;
    bool isJumping;   // ¿Está en el aire?

    double speed;     // Velocidad horizontal
    double jumpForce; // Fuerza del salto (hacia arriba)
    double gravity;   // Gravedad que lo empuja abajo
    double groundY;   // La posición Y del suelo (para que no caiga al infinito)
};
```

```
#endif // PLAYERMARATON_H
```